

CS87 Project Report: Parallelizing the RNA Secondary Structure Dynamic Programming Problem

Tillie Ferguson, Alice Huang, Katie Knox, Sojin Lim
Computer Science Department, Swarthmore College, Swarthmore, PA 19081

September 18, 2026

Abstract

There have been numerous attempts to formulate and solve the RNA secondary structure problem from biology computationally. The problem is categorized as a dynamic programming problem, since its solution can be obtained by breaking the problem down into smaller subproblems whose results are reused and combined. This nature of having numerous subproblems lends itself to parallelization, with existing approaches leveraging either the distributed memory of a network of computers or the shared memory of the GPU device. This paper explores the tradeoffs and benefits offered by these two radically different approaches in solving the secondary structure problem using Nussinov’s algorithm.

1 Introduction

In our project, we apply a parallel programming framework to the RNA secondary structure problem from biology to compare the costs and benefits of parallelizing a dynamic programming algorithm. The RNA secondary structure problem is a common application for dynamic programming techniques that seek to maximize the number of pairs formed from folding a sequence of RNA bases (A, C, G, U). For long sequences of RNA, parallelization could help improve the efficiency of finding this optimal structure that is frequently used in gene research.

We implement two parallel versions of Nussinov’s algorithm for the RNA secondary structure problem using MPI and CUDA. Though existing approaches for an MPI implementation use 4D and 8D tables in order to account for more complex folding patterns like pseudoknots in the RNA secondary structure, we design a 2D algorithm based on a block-cyclic partitioning pattern that more closely resembles Nussinov’s algorithm. We find that the CUDA shared memory parallelization produces greater speedup than the MPI implementation, though this speedup tapers off due to the high overhead in the implementation. However, we also find that the MPI column-based implementation proves to be more scalable than the CUDA implementation, as it achieves much greater speed-up when the number of processors increases. When comparing a block-cyclic partitioning of the computation table with a column partitioning, we find that a block-cyclic partitioning achieves greater parallelization of computation, yet message passing time still dominates and leads column partitioning to have better speedup overall.

Figure 1 shows how folding an RNA strand creates pairings between the bases. The number of valid pairs (A-U and G-C) is what the secondary structure problem maximizes. The optimal secondary structure is the one that minimizes the free energy in the molecule, or is the most stable. Secondary structures play a key role in gene regulation, stability, and translation, and predicting the secondary structure of a molecule

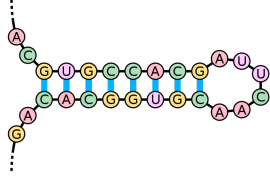


Figure 1: Folding of an RNA sequence, illustrating base pairs formed between A, U, and G, C. [8]

from a folding sequence of RNA bases provides useful information about its functionality [3].

Given the size of its search space, dynamic programming is often used to solve the RNA secondary structure problem. Dynamic programming (DP) is an optimization technique applied to recursive problems with overlapping subproblems. DP solutions typically take a “top-down” recursive or “bottom-up” iterative approach to build up a final solution. We focus on the bottom-up approach that Nussinov’s algorithm uses given it is iterative and more conducive to parallelization using CUDA and MPI. In this approach, solutions to smaller subproblems are iteratively calculated and stored in a table and used to compute larger subproblems. This eliminates the need to recalculate the same subproblems, drastically improving the time complexity. Rather than comparing every combination of base pairs, the dynamic programming solution builds the optimal secondary structure by selecting the next pairing based on the maximum pairings made in the subset of the sequence seen so far.

The focus of our project involves comparing the speedup obtained from two separate implementations of a parallel dynamic programming solution to the RNA secondary structure problem, where one is a shared memory parallelization using CUDA, and the other is a distributed memory parallelization using MPI. Our implementations are based on Nussinov’s algorithm, developed in the 1970s, which relies on the recurrence relation in Figure 2 to calculate the maximal set of valid base pairs in a given RNA strand.

$$C(i, j) = \max \begin{cases} C(i+1, j-1) + \text{bond}(\text{rna}_i, \text{rna}_j), \\ \max_{i \leq k < j} C(i, k) + C(k+1, j) \end{cases}$$

Figure 2: Algorithm to calculate the maximum secondary structure given the pairing of bases i , j [5]

The algorithm proceeds as follows. Let i and j be the indices representing the bounds of the RNA string. To find the optimal secondary structure for the entire string, we set $i = 0$ and $j = N-1$, where N is the length of the RNA sequence. Also, let $\text{bond}(\text{rna}_i, \text{rna}_j)$ denote a utility function that returns 1 if the pair $(\text{rna}_i, \text{rna}_j)$ is a valid Watson-Crick pair (A-U, U-A, C-G, or G-C) and 0 otherwise. The algorithm finds the maximal pairing for the RNA sequence by building off the maximal pairings found in subsets of the sequence. Each time we include a new pair in the set of optimal pairs, we add 1. This allows us to return the maximum number of pairs at the end of the algorithm.

Figure 2 illustrates the decision process for choosing the next optimal pair to add to the secondary structure. At each step, we take the maximum of two values. The first value is the maximum number of base pairings in the RNA strand bound by indices $i+1$ and $j-1$ (the “inner” subset of the RNA sequence excluding the current pair i, j) plus 1 if $(\text{rna}_i, \text{rna}_j)$ also forms a valid pair. This is the first case in Figure 3 of attaching a valid pair i, j onto the optimal secondary structure from $i+1$ to $j-1$.

The second value is obtained based on the intuition that instead of pairing rna_i and rna_j , we can pick an index k between index i and j . We calculate the maximal number of base pairings in the RNA substring bound by i and k , and then obtain the same information for the substring bound by $k+1$ (since k is used in the previous substring) and j . The resulting value is the maximal number of base pairings from partitioning the sequence at the index k that yields

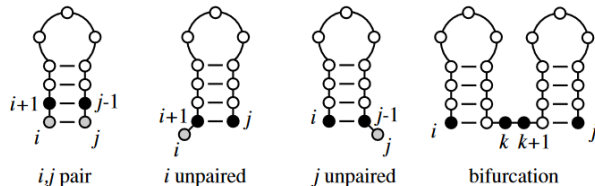


Figure 3: The four ways to maximize the secondary structure of an RNA sequence using Nussinov’s algorithm. [7]

the most pairs. This is the fourth case in Figure 3, where we split the sequence at k and combine the two optimal secondary structures from i to k and from $k+1$ to j . The second and third cases in Figure 3 show the situations where i, j does not form a valid pair, and an unpaired base is added onto the rest of the optimal secondary structure.

Because each step chooses the next pair in a way that maximizes the total number of pairs in the secondary structure, the algorithm eventually finds the optimal structure with the most base pairs possible.

2 Related Work

Researchers have been investigating the parallelization of the RNA substructure problem for many years. Much of the existing literature on parallelizing Nussinov’s algorithm focuses on shared memory approaches using CUDA [1] [2]. Distributed memory parallelization of the RNA secondary structure problem typically focuses on more recent and complex versions of the problem that include additional folding patterns such as pseudoknots [3] (which are not accounted for in Nussinov’s algorithm). This version of the problem requires 4- and 8-dimensional tables in order to store all the intermediate computations, demonstrating a much greater time and space complexity than Nussinov’s 2-d approach.

According to a 2011 paper by Chang et al., little work using GPUs to parallelize dynamic programming problems with greater than quadratic

complexity has been done up to that point (120, Chang et al, IEEE). There had been some works in the past proposing a general strategy for parallelizing Nussinov’s algorithm, but without describing the CUDA code in detail. The Chang et al. paper gives a description of the “three-kernel” strategy their implementation is based on, which divides the computation process into three phases, each executed by a different CUDA kernel, depending on the load of computation (the details of this approach will be further discussed in the following section). Later works such as the 2013 paper by Li et al. further optimize previous GPU algorithms by locating other parts in the execution flow that can be parallelized. All the different GPU algorithms, at their core, exploit the fact that given a computation matrix, each of the elements along a particular diagonal can be computed in parallel. We also note that GPU-based strategies for this problem face the problem of having a race condition that comes with shared memory usage. This means that intermediate values computed by CUDA threads/blocks must be reduced to further intermediate or final values using an efficient reduction method (or else the reduction itself becomes a performance bottleneck, as the CUDA operation for synchronizing threads is expensive).

Though there are few detailed distributed memory implementations of a parallel Nussinov’s algorithm in the existing literature, Liu and Schmidt [6] propose a new parallel design pattern called block-cyclic based wavefront, which can be applied to dynamic programming problems like the RNA secondary structure problem to balance the load assigned to different processors in an MPI implementation. When parallelizing dynamic programming problems, processors often perform computations in “wavefronts” or groups where each new wavefront depends on calculations from previous wavefronts; processes therefore must synchronize before moving onto the next wavefront together. While it may be effective in some problems to partition the compu-

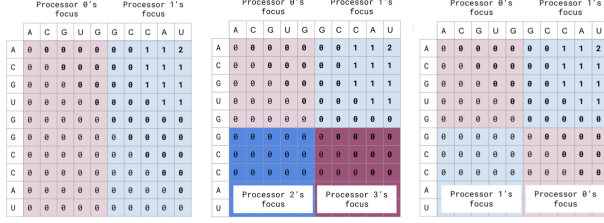


Figure 4: Representations of the three different design approaches used to solve the RNA Subsequence problem in parallel using a distributed architecture. a) Column partitioning, where process 1 is doing the majority of the work b) Block partitioning, where process 1 is working for the entirety of the computation time and process 2 never works, and c) Block-cyclic partitioning, where the load is balanced more evenly across processors

tation matrix such that each process is assigned adjacent columns, the nature of the wavefronts in a problem like the RNA secondary structure problem leads this approach to generate an uneven workload across processes. This can be seen in Figure 4a, where process 0 finishes its partition of the table before process 1, which is assigned more diagonal wavefronts to complete.

The block-cyclic-based distribution proposed distributes cells to processes in a cyclic way, rather than splitting the matrix by adjacent groups of columns. This is illustrated in Figure 4c, where each process is assigned two partitions of cells, and as the computation progresses in wavefronts through the upper triangular, the work is shared between the two processes. Though this helps to balance the load across processes, it also creates a trade-off because the number of divisions of the matrix also increases. Liu and Schmidt show that for Nussinov’s algorithm, using the block-cyclic-based method achieves much greater speedup than using the column-based method, despite the increases in communication overhead due to a greater number of divisions.

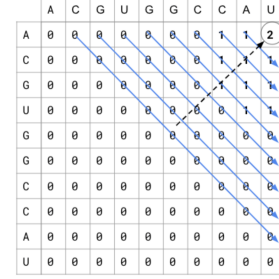


Figure 5: The direction of computation in Nussinov’s algorithm.

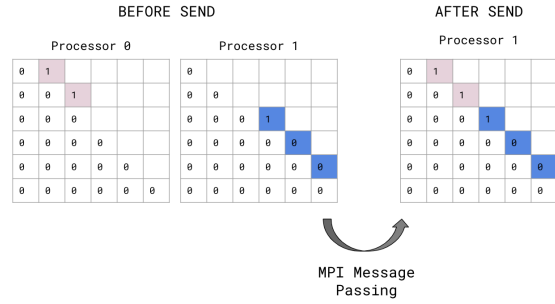


Figure 6: Representation of message passing in column-based partitioning implementation of Nussinov’s algorithm.

3 Our Solution

In Nussinov’s algorithm, the upper triangular of a 2D table is iteratively filled in order to find the maximum pairing for the RNA sequence. Computation proceeds in diagonals or “waves” toward the upper rightmost corner of the table. This is illustrated in Figure 5, where the solid blue arrows show the direction of computation along each diagonal and the dashed black arrow shows the order in which the diagonals are computed.

Our solution consists of three main components based on Nussinov’s algorithm: a sequential implementation in C, a shared memory implementation using CUDA, and a distributed memory approach using MPI.

We explore three versions of an MPI implementation that partitions the table in different

ways among the processors. Given the large amount of memory resources on a distributed architecture, each process in our MPI implementation stores a copy of the entire board and sends its computed sections of each diagonal to the other processes. Our initial solution uses column partitioning, where each process is assigned groups of adjacent columns in the table. For an RNA sequence of length N and P processors, each process is responsible for N/P columns. Each process computes the values in the diagonal wavefront that fall in the column it is responsible for. Since there are only backwards dependencies, it is only necessary for each process to send to processes that have a rank larger than its own. This is demonstrated in Figure 6, where each process computes the values of the wavefront it is responsible for and then Process 0 sends its values to Process 1. Computation dominates in this implementation because the upper triangle is not evenly partitioned, and the most significant process rank ends up doing the bulk of the computational load.

Another implementation we explore in this study is a block partitioning design. Processes are assigned to “block” partitions in row-major order, where each block is an N/P by N/P sized section of the table. Figure 4b shows an example of this distribution for four processes. This implementation is constrained to problem sizes that can be partitioned evenly into P even boxes; which manifests in a constraint of problem sizes that are evenly distributed for $P = 4, 16, 64$, etc. There are less stringent constraints for the other implementations. [come back to this? Or don’t include] Each block computes values when the diagonal wavefront is “in bounds,” and sends those values upwards to processes that are higher in the upper triangular distribution. This distribution of work is still unbalanced, given processor 1 has the most rounds of computation to complete, while processor 2’s block lies completely outside the upper triangular (no rounds of computation).

Our third implementation of this algorithm us-

ing a distributed system design is block-cyclic partitioning. In this design, processes are cyclically assigned to blocks of the table to compute such that the computationally heavy portions of the table are more evenly distributed. Figure 4c shows an example of what this would look like for 2 processes. Note that this distribution is the same as that of the block partitioning (Figure 4b), but it is able to partition the same amount of sections among half as many processes (formally, the square root). In this implementation, each process iterates through their “blocks,” calculating any values that fall in bounds of the current wavefront and sending “up” to processes with a more significant rank. Figure 7 has an example of what the distribution looks like for more processors. In this approach, the problem size N must be divisible by the number of processors P .

In all three implementations, all the processes simultaneously loop through the upper triangular of their copy of the table in diagonal waves. When a process reaches cells that fall within the partition(s) it is responsible for, it computes the maximum pairings for the subproblems represented by those cells using the algorithm described in the Introduction section and Figures 2 and 3. The process then sends the values it computed to other processes that still need them, so all processes can update their version of the table. In this way, processes compute each wavefront in parallel until the most significant process ultimately computes the maximal pairings for the problem as a whole. As a result, we apply the advantages of parallelization to Nussinov’s dynamic programming algorithm for the RNA secondary structure problem.

In our CUDA implementation, we follow the general strategy described by Chang et al. As dictated by Nussinov’s algorithm, the value of each element e in the matrix requires taking the maximum of the vector sum of 1. the row vector starting at the main diagonal and up to e and 2. the column vector starting below e and all the way down to the main diagonal.

The process of obtaining the vector sum can

0	1	2	3
3	0	1	2
2	3	0	1
1	2	3	0

Figure 7: The partitioning of processors in a block-cyclic organization for 4 processors. The blocks which will participate in computation fully or partially are shown in bold.

	A	C	G	U	G	G	C	C	A	U
A	0	0	0	0	0	0	0	0	0	0
C	0	1	0	0	0	0	0	1	0	0
G	0	0	0	0	0	0	1	1	0	0
U	0	0	0	0	0	0	0	0	0	0
G	0	0	0	0	0	0	0	0	0	0
G	0	0	0	0	0	0	0	0	0	0
C	0	0	0	0	0	0	0	0	0	0
C	0	0	0	0	0	0	0	0	0	0
A	0	0	0	0	0	0	0	0	0	0
U	0	0	0	0	0	0	0	0	0	0

Figure 8: Computational dependencies of the Nussinov computation matrix. To obtain the value whose cell is colored in green, each element in the row vector (cells in blue) is added to its corresponding element in the column vector (cells in red), in the direction of the arrows.

be parallelized, as all of the values in the row and column vector have been calculated in the previous steps. With its block and thread constructs, CUDA provides multiple ways to parallelize. However, as with all CUDA programs, attention had to be given to the way in which we partition the board into grids of blocks and into blocks of threads, for two reasons. First, the GPU architecture of the machine and CUDA software place limits on the maximum size of blocks and grids that can be configured on kernel launch. Secondly, the amount of computation required to obtain the value of a particular element in the matrix depends on which diagonal

the element is located.

For the above reasons, our initial implementation divided the computation into three different phases. The first phase is associated with many elements along the diagonal and light computation per element. Therefore, in this phase, each element gets a single thread working to compute the value of the element. In the second phase, characterized by a moderate amount of elements along the diagonal and moderate amount of computation per element, we assigned a block of threads to each element. In the final phase, there are few elements along the diagonal but the amount of computation done per element is heavy. Because of this, we assign an entire grid to each element.

As we began to test our code, however, we observed that the three-stage approach is not easily scalable because of the restriction to 1024 threads per block on CUDA version 6.1. Because the initial phase allocates one thread per element on the middle diagonal, if the RNA strand itself is longer than 1024 elements, we ran into memory access errors. We got around this by implementing a check to determine if the problem size is small enough for the initial diagonal to be computed using one thread per block, and if otherwise, we compute the whole grid using the final-stage kernel, which allocates a grid per element. For the purposes of comparing CUDA speed-up to MPI speed-up, our group decided to test very large samples, ranging from 3,000 - 12,000 nucleotides in length. This means that our code effectively computes the maximal number of pairings using the final stage (creating a whole grid per element). The three-stage method may be best suited for imitating real-life purposes, seeing as the average RNA strand in nature is approximately 200 nucleotides [4].

In the second and third phase, we encounter a race condition where each thread tries to read and write to the same offset in the matrix (the position of the element of interest). In order to obtain the correct maximum value, we must have the threads each write to a different offset in a

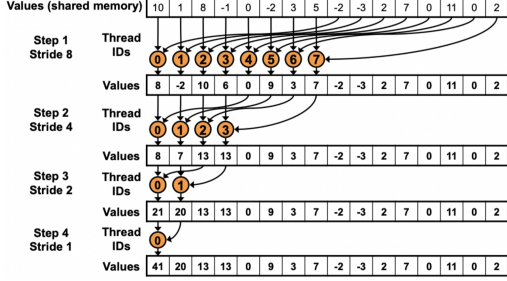


Figure 9: The Sequential Addressing Reduction Method. [5]

separate array of shared memory, and reduce the values in the array to a final maximum value. Similarly, in the final phase, we allocated shared memory visible to all the blocks in a particular CUDA grid. The threads within each block reduce and yield a local maximum value, which is then written to shared memory and subsequently reduced to a global maximum value among all the blocks. We explored two different methods of reduction, the second method building on the first. The first of the reduction methods, called “sequential addressing,” has $\log_2 n/2$ number of steps where n is the length of the shared memory array to be reduced. At each step, the thread with thread id i computes the maximum of the value at index i of the array, and the value at index $i + s$ where s is the ‘stride’ associated with the step [5]. The stride value starts at $n/2$ and is subsequently decreased by a factor of 2.

Our improved reduction method uses the fact that NVIDIA GPUs execute warps of 32 threads using SIMT (single instruction, multiple threads), on top of sequential addressing. In the sequential addressing method, we used a for loop to progress through the steps in the reduction, decreasing the stride value at every step, and synchronizing at the end of the step. However, once the stride value reaches 32, the threads will be executed using the same instruction and therefore does not require synchronization (which is an expensive CUDA operation). Therefore, the last 32 threads can be unrolled manually in a separate function (Figure 10).

```
__device__ void warpReduce(volatile int* sdata, int tid) {
    sdata[tid] += sdata[tid + 32];
    sdata[tid] += sdata[tid + 16];
    sdata[tid] += sdata[tid + 8];
    sdata[tid] += sdata[tid + 4];
    sdata[tid] += sdata[tid + 2];
    sdata[tid] += sdata[tid + 1];
}
```

Figure 10: The The template for reduction that ‘unroll’ the last warp.

This saves the overhead for exactly 5 iterations of the for loop and the synchronization operation. In the following section, we briefly discuss the performance impact of warp unrolling.

4 Results

Our goal with this project was to determine the efficacy of parallelizing the dynamic programming approach to the RNA folding problem, and our secondary goal was to determine if a shared-memory or message passing implementation would be most effective for this type of problem. As discussed, we split into two groups, and implemented the shared-memory version of Nussinov’s algorithm using CUDA, and the message passing version using MPI. In order to compare efficacy, we measured the total computation time of the sequential, CUDA, and MPI programs. For all three versions of the algorithm, we took measurements for problem sizes of 3840, 7680, 10,240 and 12,288 due to the constraints for partitioning the table in the MPI implementations. For MPI, we ran at least two experiments for each problem size for varying numbers of processors and for each of the three partitioning methods (column, block, and block-cyclic). For CUDA, we ran three experiments for each problem size.

We constructed RNA sequences of different problem sizes for testing using repeating sequences of ‘CG’ bases. We used this approach given our project focus on the computational tradeoffs of parallelizing the RNA secondary structure problem rather than the applicability of our solution to predicting secondary structures

for real sequences from biology. The MPI implementations for problem sizes 3840 and 7680 were run on the Swarthmore CS Department computers, while the tests of size 10,240 and 12,288 were run on the XSEDE system. Below, we discuss both the high-level results obtained from averaging the execution time for each implementation, as well as the insights gained from looking at more specific breakdowns. The CUDA implementation was run on the Swarthmore CS Department computers for all problem sizes.

Looking at the average execution time and speed-up of all three implementations confirms our hypothesis that CUDA would be the most efficient implementation. All three implementations exhibit a roughly linear increase in execution time, with the sequential implementation exhibiting the highest slope and the CUDA version the smallest slope. Speed-up measures the ratio of the sequential execution time to the parallel execution time; an ideal speed-up of the parallel versions would be a linear slope.

The speed-up of the CUDA implementation is better compared to MPI on average, and also exhibits a greater slope for the smaller RNA sizes. This shows that CUDA is much more efficient than the sequential and MPI versions at smaller values, but as the problem size increases, the slope of the speed-up tapers off. CUDA is likely faster because it relies on shared memory access while MPI requires message passing, and this algorithm relies on a lot of shared memory access in order to compute the vector sums. The CUDA speed-up tapers off due to the high overhead involved in our CUDA implementation, seeing as all of the problem sizes skip the initial phase (see above CUDA implementation details), and there is a whole kernel launch and grid allocation for each of the RNA-grid elements. Additionally, we perform many thread synchronizations within our kernel code for reduction operations, and these synchronizations mean that each parallel phase takes as long as the slowest thread.

After observing the limitations of CUDA speed-up, we decided to compare it to a break-

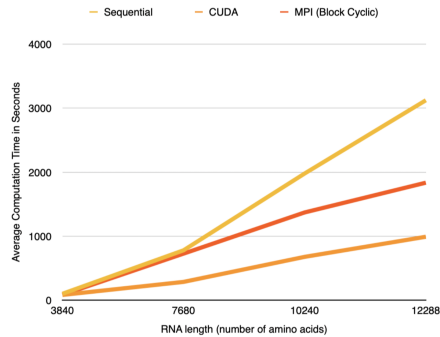


Figure 11: Overall time comparison of sequential versus parallel code.

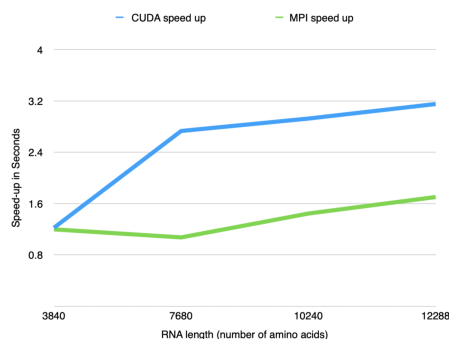


Figure 12: Speed-up of CUDA versus MPI implementations (block-cyclic).

down of the MPI speed-up (using column partitioning this time) over different numbers of processors. In our overall measurement of the MPI speed-up, we averaged the execution time across all numbers of processors. When we examine this breakdown, however, we see that while CUDA exhibits greater speed up when compared to MPI on 2 - 8 processors, for greater numbers of processors, MPI is more efficient. CUDA is thus limited in its scalability because it is executed on only one GPU, whereas the ability to increase the amount of parallel processing in MPI offsets the initial price of message-passing.

We also performed tests comparing the speed-up for the three different partitioning patterns for the MPI implementation in order to determine whether a more balanced workload across processors can achieve greater speed-up. In the-

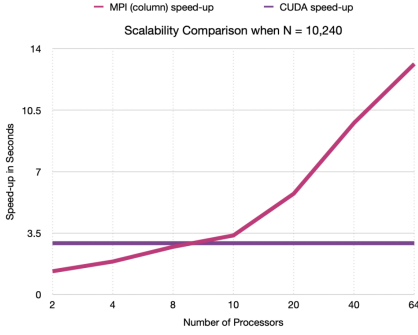


Figure 13: Speed-up of CUDA versus MPI implementations (column).

ory, if computation is distributed more evenly across processors, more computation can occur in parallel. This is the most efficient use of parallelization techniques that takes full advantage of having multiple processors. Figure 14 illustrates the speed-up comparison for the column and block-cyclic partitioning patterns. We observe no significant speed-up in the block-cyclic version, while the speed-up for the column version is much greater (especially once the number of processors reaches 10). This is an unexpected result, given the block-cyclic partitioning is the most balanced among the three versions and the column partitioning is the least balanced.

However, our experiments indicate the total time required for the block-cyclic partitioning is dominated by the message passing required to exchange cell values computed along the diagonals in each process’ blocks. This can be seen in Figure 15. For the block-cyclic approach, computation time dominates at first, but communication time quickly takes over with even the addition of two more processors for a total of four processors. As originally expected, Figure 14 shows that the block-cyclic distribution of work achieves better parallelization of computation than the column-wise partitioning does. However, the buffer sent in the cyclic approach is significantly larger to account for the much larger amount of potential values that can be calculated by a process during a single wavefront. This

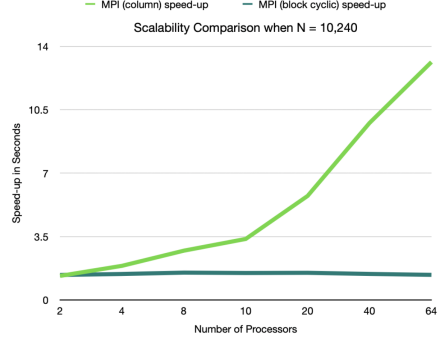


Figure 14: Speed-up of column-based partitioning and block-cyclic partitioning implementations.

significantly increases the communication time and causes the performance of the block-cyclic implementation to lag in comparison to that of the column-wise implementation. With column partitioning, computation time dominates even when the number of processors reaches 64. These results indicate that despite the benefits of using more processors to perform computation in parallel in a block-cyclic partitioning pattern, the increased amount of message passing required and the large amount of data sent prevents this approach from achieving greater speed-up than a column partitioning implementation that uses a less efficient partitioning of table cells to processors.

Although it was not the main focus of our project, it was interesting that the CUDA implementation saw a significant performance improvement from using the warp unrolling strategy in addition to the sequential addressing reduction method, as Figure 10 shows. As we scaled up the length of the RNA sequence, manually unrolling the last warp brought close to a 50% reduction in total execution time from just relying on sequential addressing. This result demonstrates the importance of reduction in shared-memory parallelism, as it ensures performance of the solution which could be brought down by the need to prevent race conditions.

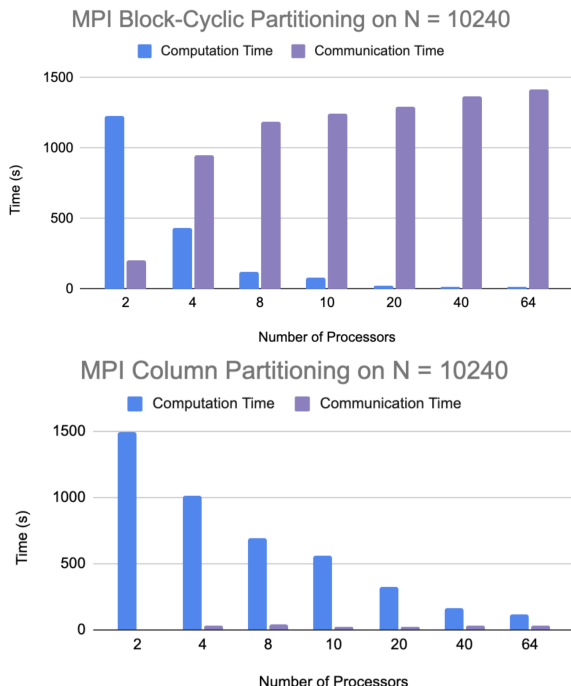


Figure 15: Comparison of time spent on computation vs communication (message passing) in MPI a) block-cyclic implementation and b) column implementation on a problem size of 10,240 base pairs.

5 Conclusions and Future Directions

Our project meaningfully compares radically different parallelization techniques for the dynamic programming solution to the RNA-folding problem. The results of our experiments show that the shared-memory CUDA implementation exhibits greater speed-up compared to the distributed-memory MPI implementation for solving the RNA secondary structure problem using the Nussinov algorithm. Additionally, the speed-up of CUDA is greater for smaller values of RNA length, and because RNA sequences found in nature are around 200 nucleotides in length [4], this parallelization would be best suited for real life applications.

Importantly, we gained insight not only about the overall efficiency of each technology, but by

Sequence Length	Sequential Addressing Execution Time (s)	Sequential Addressing + Warp Unrolling Execution Time (s)
3780	81.58	57.03
7680	285.14	239.53
10240	524.36	270.94

Figure 16: Execution times of the CUDA implementation using the sequential addressing and warp unrolling reduction methods.

comparing shared and distributed memory implementations side by side, we also gain a better understanding of their representative strengths and weaknesses. However, we cannot generalize and say that all CUDA or shared-memory implementations are better than MPI or message-passing implementations. There exist numerous other parallelization strategies for both CUDA and MPI, and our experimentation showed us that partitioning and reduction techniques have a huge influence on the overall performance of the technology. Therefore, a possible next step would be to optimize the respective parallel implementations using different strategies. For instance, we prioritized working on different reduction strategies with our CUDA implementation because we saw this as a significant bottleneck, but it would have also been interesting to examine the effects of a different partitioning method, such as the block strategy. This would have been particularly interesting given that the three-phase strategy essentially because a one-phase, maximalist strategy for large problem sizes. It would also be interesting to see a hybrid parallel implementation where each node in the distributed system uses CUDA to speed up their individual computations. This would help overcome the limitations of both technologies, such as the communication overhead of MPI, or the danger of race conditions in using CUDA). As we showed above, while message-passing does introduce an expensive overhead, it can be more efficient than CUDA when taking advantage of more processors and using column partitioning.

6 Meta-discussion

The MPI team struggled to develop our initial approach, given the general lack of preexisting literature and resources on an MPI solution to this problem. The implementation for the block and block-cyclic approaches required thoughtful computational thinking about how to orchestrate the partitioning and calculate each processor’s appropriate bounds. Our testing yielded unexpected results, and we had to revisit our implementations and assess what the cause was of the varied performances.

For the CUDA team, the most difficult part was working around the CUDA thread-block size limit (it took us a long while to even realize that we were getting errors because our blocks contained more than 1024 threads, because the kernels had seemed to launch successfully despite this issue). Although we planned in our proposal to implement and test a hybrid parallelization technique that utilizes both distributed and shared memory, we were unable to get to that part due to limited time. We pivoted from a hybrid approach in favor of a more comprehensive study of the problem’s behavior and solutions on the two architectures, studying several implementations in CUDA and MPI.

References

- [1] Ra’ed M. Al-Khatib, Rosni Abdullah, and Nur’Aini Abdul Rashid. A Comparative Taxonomy of Parallel Algorithms for RNA Secondary Structure Prediction. *Evolutionary Bioinformatics*, 6:EBO.S4058, 2010.
- [2] Karl-Eduard Berger and Francois Galea. An Efficient Parallelization Strategy for Dynamic Programming on GPU. In *2013 IEEE International Symposium on Parallel Distributed Processing, Workshops and Phd Forum*, pages 1797–1806, 2013.
- [3] P. Evans E. Snow, E. Aubanel. Dynamic Parallelization for RNA structure comparison. In *2009 IEEE International Symposium on Parallel and Distributed Processing*, pages 1–8, 2009.
- [4] Alessandro Fatica and Irene Bozzoni. Long non-coding RNAs: new players in cell differentiation and development. *Nature Reviews Genetics*, 15(1):7–21, January 2014.
- [5] Mark Harris. Optimizing parallel reduction in CUDA, 2007.
- [6] Weiguo Liu and Schmidt. Parallel design pattern for computational biology and scientific computing applications. In *2003 Proceedings IEEE International Conference on Cluster Computing*, pages 456–459, 2003.
- [7] Bayesian Neuron. Nussinov algorithm to predict secondary rna fold structures, 2019.
- [8] Wikipedia, the free encyclopedia. Stem loop, 2006.